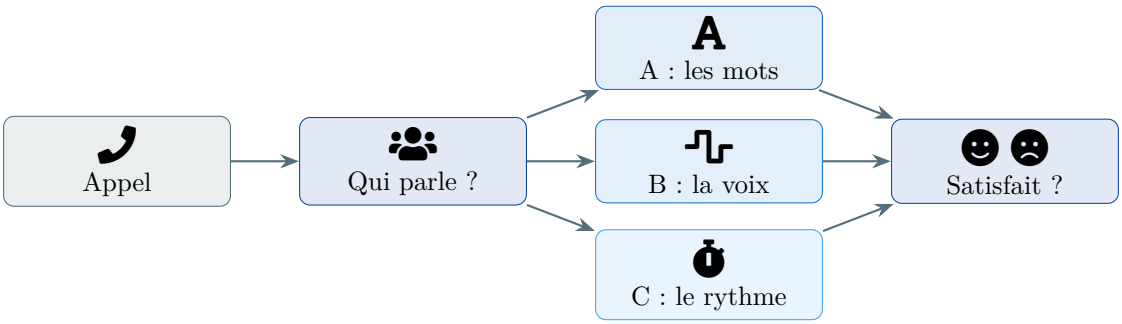
École d'automne IA, Alsace Tech

Hackathon voix du client

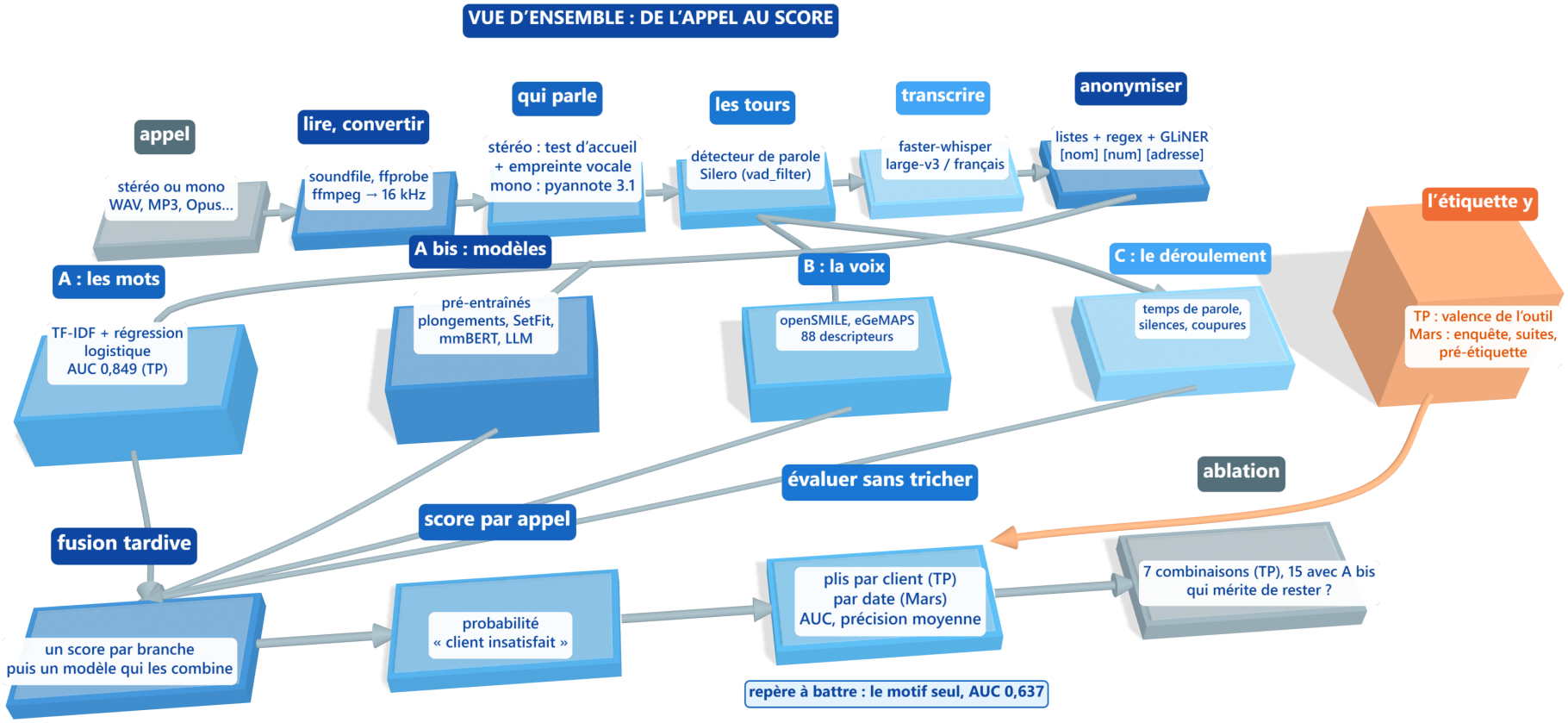
Fiches 1 à 4

Ton GPU gratuit, puis les séances 1 à 3 du TP



Objectif : allumer un GPU gratuit sans gaspiller ton quota, puis vérifier tes données avant d'entraîner quoi que ce soit.

Vue d'ensemble : les modèles et la méthode



1. Préparer

Lire et convertir le son, savoir qui parle, découper les tours, transcrire, puis masquer noms et numéros **avant tout modèle**.

2. Des branches indépendantes

A les mots (TF-IDF), **A bis** les modèles pré-entraînés, **B** la voix, **C** le déroulement : chacune donne un score, une fusion tardive les combine.

3. Juger honnêtement

Une étiquette auditée, un découpage sans tricher (par client, par date), un repère à battre (0,637), puis l'ablation.

1. Ton GPU gratuit en ligne

On travaille sur KAGGLE. Colab sert seulement de secours.

Kaggle

- **2 × T4**, 16 Go chacune
- environ **29 Go** de RAM
- **12 h** par session
- environ **30 h** de GPU par semaine
- tourne **en arrière-plan** (« Save & Run All »)
- garde tes fichiers dans `/kaggle/working`

Colab gratuit (secours)

- **1 × T4**, 16 Go, **pas garantie**
- environ **12,7 Go** de RAM
- **12 h** au plus, souvent moins
- quota **inconnu**, il change
- l'onglet doit **rester ouvert**
- fichiers **perdus** sans copie vers Drive

À faire AVANT le hackathon

1. Créer un compte **kaggle.com**
2. **Vérifier ton téléphone** (sinon : ni GPU ni Internet)
3. Compte **Hugging Face** + jeton de lecture
4. Accepter les conditions du modèle de **diarisation**

Allumer le GPU, en 4 clics

1. **Create → New Notebook**
2. Panneau de droite : **Session options**
3. **Accelerator → GPU T4 x2**
4. **Internet On**

Ta première cellule : vérifie ta machine

```
!nvidia-smi
!free -h
import torch
print(torch.cuda.device_count())
print(torch.cuda.get_device_name(0))
```

Tu dois voir :

- deux lignes `Tesla T4`
- `2`
- `Tesla T4`

Tu vois `0` ?
Reprends l'étape 3, puis vérifie ton téléphone.

2. Les 5 règles d'or du GPU

Transcris UNE fois.
Sauvegarde le résultat, puis travaille toujours sur les fichiers sauvegardés.

Arrête ta session.
Une session ouverte mange ton quota, même si rien ne tourne.

Calcul long ?
« Save & Run All » : ça tourne seul, tu peux fermer l'onglet.

T4 = `fp16`.
Jamais `bf16`, jamais FlashAttention.

2 × T4 = deux cartes de 16 Go, PAS une carte de 32 Go.
Pour aller deux fois plus vite : coupe ta liste d'appels en deux, un processus par carte (code ci-dessous).

Utiliser les deux GPU : un processus par carte

```
import subprocess, os
procs = []
for gpu in (0, 1):
    env = dict(os.environ, CUDA_VISIBLE_DEVICES=str(gpu)) # ce processus ne voit qu'
        UNE carte
    procs.append(subprocess.Popen(
        ["python", "transcrire.py", "--part", str(gpu), "--nparts", "2"], env=env))
for p in procs:
    p.wait() # on attend les deux moitiés
```

Ça plante ? Cherche ton message ici

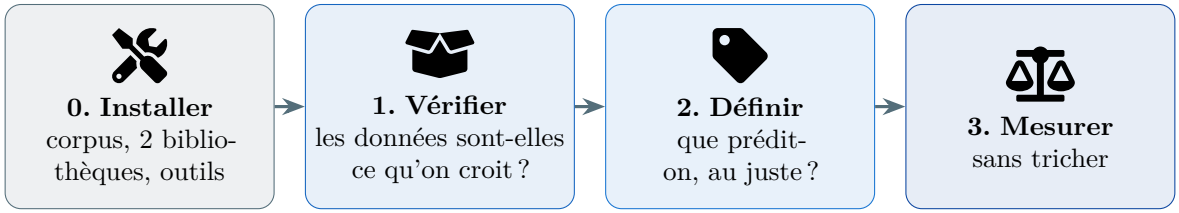
Tu vois…	Tu fais…
`device_count() == 0`	→ Accelerator : GPU T4 x2. Téléphone vérifié ?
`pip : erreur réseau`	→ Session options : Internet On
`CUDA out of memory`	→ `batch_size=4`
`bfloat16 / bf16`	→ remplace par `fp16=True`
`flash_attn / flash_attention_2`	→ supprime l'option `attn_implementation`
`diarisation : 401 / gated / token`	→ jeton Hugging Face + conditions acceptées sur la page du modèle
`quota épuisé`	→ un coéquipier lance le calcul depuis son compte

Une question ? Lève la main, l'encadrante passe.

Fiche 2 : séance 1

Avant d'entraîner quoi que ce soit, on vérifie.

3. La séance en une image



Comme une recette de gâteau : vérifier que le sachet « sucre » contient du sucre, choisir quel gâteau, et décider comment on saura s'il est réussi. *Avant* d'allumer le four.

4. Partie 0 : mise en route

Exécute les 3 cellules, sans rien modifier

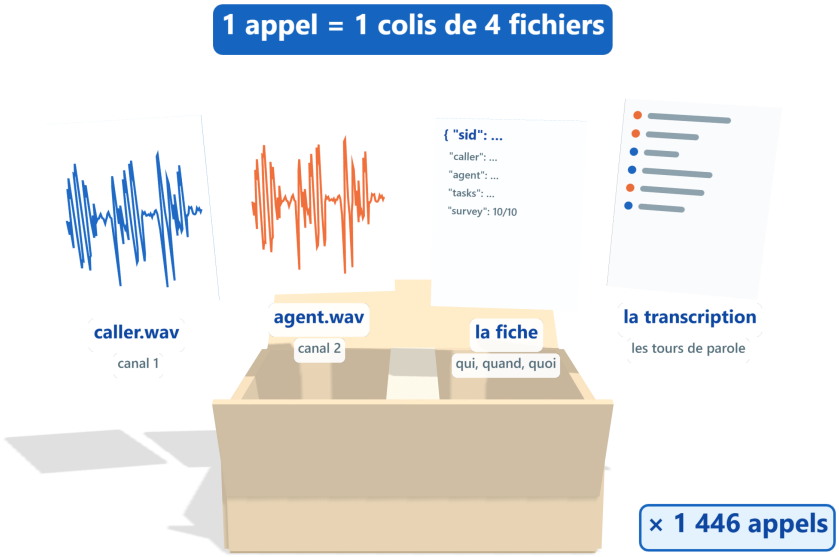
- **0.1** télécharge le corpus : environ 650 Mo, 3 Go sur le disque. Quelques minutes, sans affichage.
- **0.2** installe `opensmile` (la voix) et `jiwer` (les erreurs de transcription).
- **0.3** charge les outils du TP, dont `verifier`.

Tu dois voir : `1446 appels disponibles`, puis `Outils chargés.` et la liste des versions.

`/kaggle/working` = le **cartable**, pesé à chaque sauvegarde. `/kaggle/temp` = la **table**, nettoyée le soir. Le corpus va sur la table.

`[À revoir]` = la maîtresse écrit « à revoir » dans la marge, avec un **indice**, au lieu de déchirer la copie. Lis l'indice !

5. Partie 1 : contrôle à la réception



Un canal

une ligne enregistrée à part : canal 1 = client, canal 2 = conseiller (en principe).

deux talkies-walkies, deux cassettes

Un tour

un moment où une seule personne parle.

chacun son tour d'avoir la balle

Un faux tour

un souffle ou un clic détecté comme un tour : `[noise]`.

la lampe qui s'allume quand passe un chat

1.1 Deux tables : `appels` (1 ligne par appel) et `tours` (1 ligne par tour)

Tu dois voir :

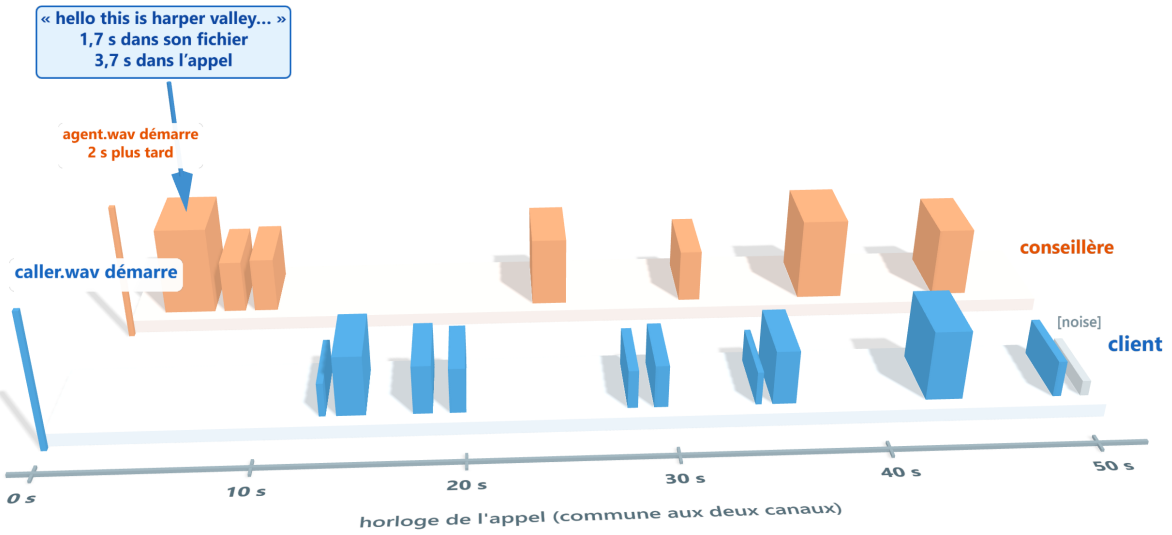
1 446	25 730	56 s	9 s	55
appels	tours	médiane d'un appel	d'attente (médiane)	clients

1.2 Le client est-il toujours sur le canal 1 ?

Pour chaque appel : le canal le plus fréquent des tours du client. Compte ceux qui ne sont pas 1.

Tu dois voir : 37 appels inversés. Ce n'est pas ton code : c'est dans la base.

1.3 Les deux horloges : l'exercice le plus important



`position_s` : où est la voix dans son fichier

pour **découper** l'audio (branche B)
➔ c'est `start_ms`

`debut_s` : quand dans l'appel

pour **ordonner** les tours (branche C)
➔ c'est `offset_ms`

deux chronomètres : celui de la conseillère est lancé 2 secondes après celui de la cliente.

Tu dois voir : écart médian de **2,01 s** sur le canal 2, 0 sur le canal 1 ; la voix est à `start_s` dans **96 %** des 1 000 tours testés.

C'est l'inverse de ce que dit la documentation. En la croyant, tu découpes le conseiller 2 s trop tard dans 70 % des appels, et ton code ne t'affiche **aucune erreur**.

1.4 Dans quel fichier est la voix du client ?

Dans les 37 appels inversés, mesure l'énergie de chaque tour du client dans `caller` et dans `agent`. Le fichier le plus fort contient sa voix.

Tu dois voir : 162 tours testés, `agent` plus fort dans **89 %** des cas.

Règle : le fichier se choisit par le **canal**, le rôle se lit dans la transcription.

`canal 1` ➔ `caller/`
`canal 2` ➔ `agent/`

deux tiroirs avec des étiquettes collées à l'envers : on ouvre et on regarde.

1.5 Regarde et écoute (rien à coder)

Change `sid_exemple` (`appels.index[10]`, `[200]`…) et écoute l'autre canal. Toutes les zones colorées contiennent-elles de la parole ?

1.6 Nettoyer et écarter le bruit

`nettoyer` : minuscules, sans `[balises]` ni `<unk>`, seulement lettres, apostrophe, espaces. `parole` = la transcription **automatique** nettoyée n'est pas vide.

✔ **Tu dois voir : 14,4 %** des tours du client sans parole.

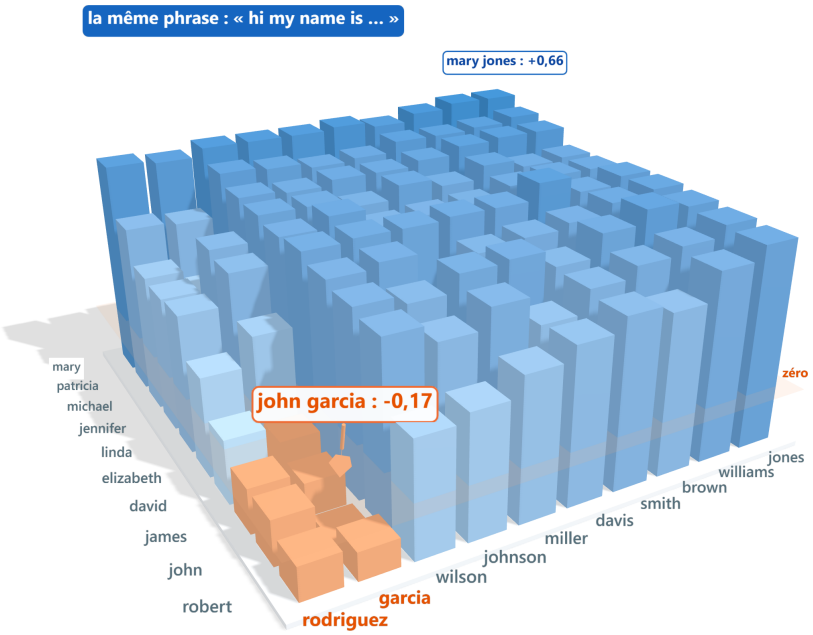
6. Partie 2 : l'étiquette

valence $= p_{\text{pos}} - p_{\text{neg}}$ → $y = 1$ si l'appel est dans le **quart le plus bas**

Un thermomètre d'humeur, pas l'humeur. $y = 1$ ne veut pas dire « client mécontent », mais « l'outil l'a trouvé moins positif que les autres ».

2.1 à 2.4 Construire, puis auditer l'étiquette

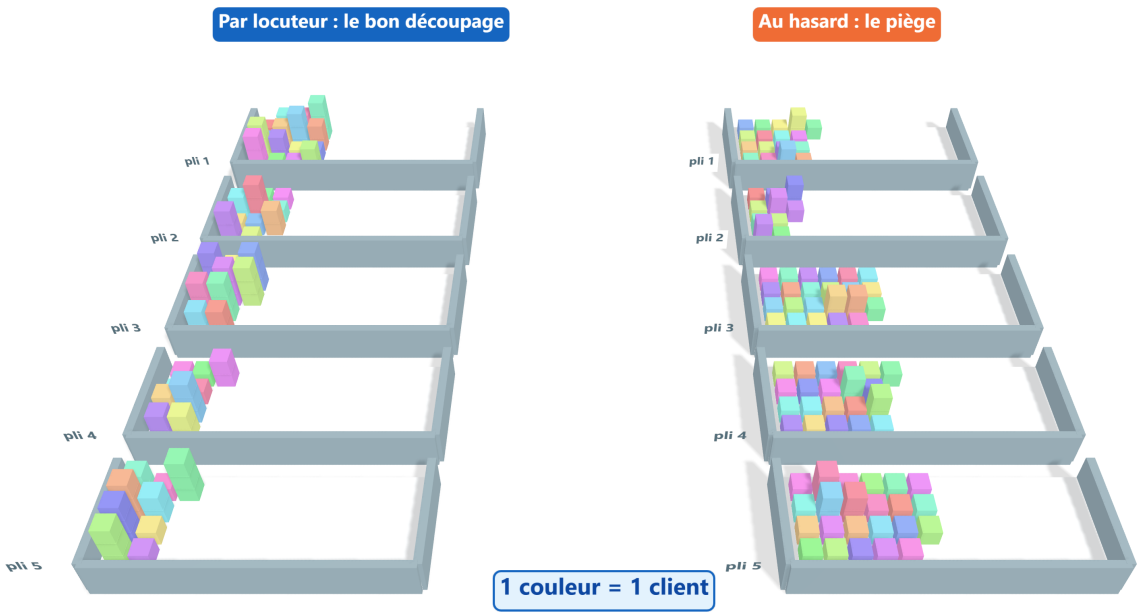
2.1	construire `valence`, `valence_client`, `seuil`, `y`	✔ seuil **0,039** ; **362** appels à 1
2.2	lire les 15 tours les plus négatifs et les 15 plus positifs	✔ **3 322** tours candidats
2.3	part de $y = 1$ par motif et par client	✔ **8** motifs, **45** clients
2.4	la même phrase « hi my name is… » selon le nom	✔ **386** présentations



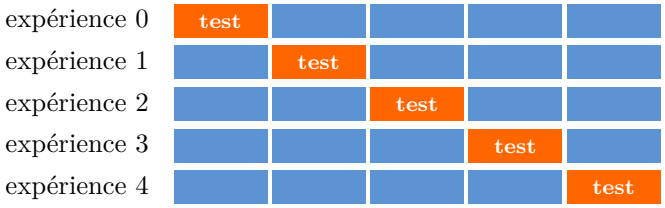
La même phrase, deux notes. « John Garcia » : −0,17. « Mary Jones » : +0,66. Le nom change la note de l'outil. → On **anonymise avant** tout outil de sentiment.

Retiens : un nom dans le texte, même celui d'un personnage fictif, devient un biais. Dans notre projet, on masque les noms **dès le départ** : `[nom]` pour tout le monde.

7. Partie 3 : mesurer sans tricher



Examen et révisions : à l'examen, pas d'exercices de la même personne qu'en révision, sinon on reconnaît l'écriture au lieu de comprendre. Ici : **tous les appels d'un client du même côté**.



→ chaque appel est testé **une fois**, par un modèle qui ne l'a jamais vu : ce sont les **scores hors pli**

3.1 Les plis

```
plis = list(StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=0)
            .split(appels, y, groups=appels.id_client))
```

Tu dois voir : 5 plis de **273 à 321** appels, **9 à 16** clients chacun, aucun client des deux côtés.

AUC

On tire un appel à 1 et un appel à 0 : chance que le modèle classe le 1 plus haut.
0,5 = pile ou face **1** = parfait

le jeu des deux enveloppes

Précision moyenne

Regarde surtout le **haut de la liste** : les appels qu'on remonterait à un superviseur.
0,25 = hasard

3.2 Le repère à battre : le motif seul

Tu dois voir :

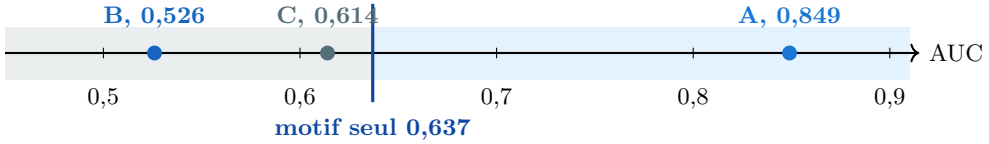
0,637 AUC	**0,349** précision moyenne

Ta barre, c'est 0,637, pas 0,5. Une branche à 0,62 n'a rien appris : le motif seul fait mieux.

deviner la météo en disant « comme aujourd'hui » : le vrai météorologue doit faire mieux.

La barre « motif seul » que tu verras avant la branche A

Ce n'est pas une branche. C'est un modèle qui ne connaît que la raison de l'appel (« pay bill », « get branch hours »…), recalculé dans les mêmes plis et posé à côté des branches pour servir de ligne de lecture. Il atteint 0,637 parce que certains motifs sont plus pénibles que d'autres : 47 % d'appels étiquetés 1 pour « pay bill », 11 % pour « get branch hours ».



À gauche de la ligne : la branche n'a rien appris de plus que le motif. À droite : elle apporte de l'information sur la tonalité.

En fin de séance, tu as :

2 tables propres
`appels`, `tours`

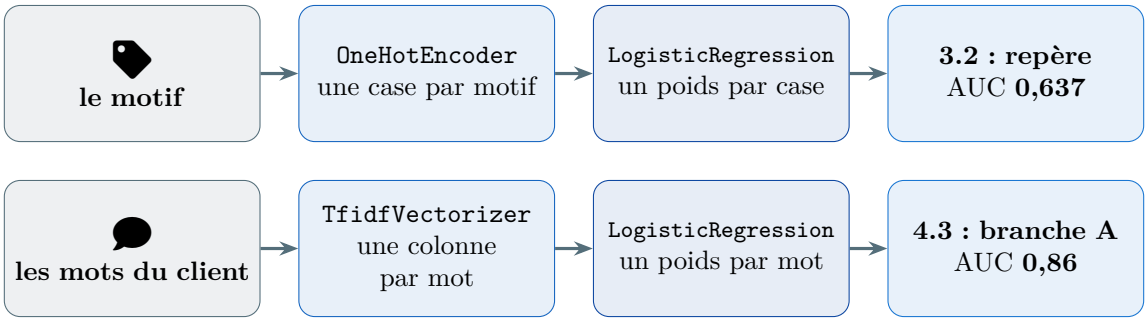
1 étiquette
dont tu connais les défauts

1 protocole
plis par client, repère 0,637

Fiche 3 : séance 2

Les mots : ton premier classifieur.

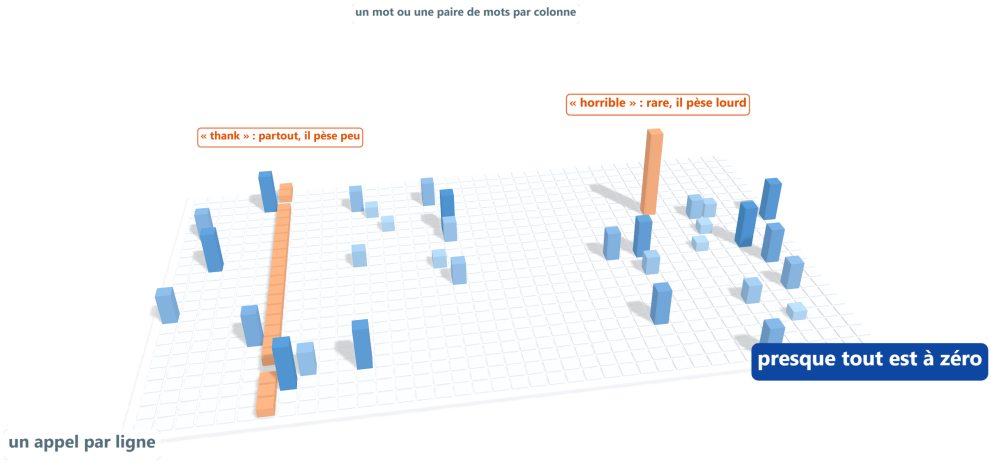
8. Deux modèles, les mêmes briques



`OneHotEncoder` : **une case cochée par motif**, comme sur un formulaire. Un modèle ne sait pas lire « pay bill », il sait lire des 0 et des 1.

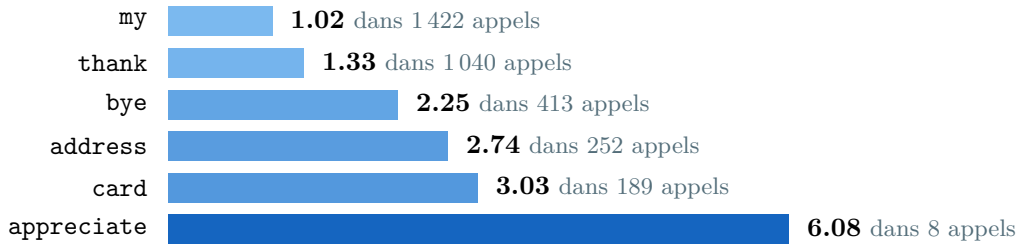
`LogisticRegression` : **une balance à mots**. Chaque mot pose un petit poids d'un côté ou de l'autre ; la balance penche, et on lit un pourcentage entre 0 et 100.

9. TF-IDF : un appel devient une ligne de nombres



TF-IDF = **TF** (le mot est-il *ici* ?) × **IDF** (est-il *rare* ailleurs ?)

IDF sur les 1 446 appels du TP : plus rare = plus lourd



Un sac de mots : on vide la phrase dans un sac et on compte, en donnant plus de poids aux mots rares. Le sac sait que « horrible » est dedans, pas qu'on a dit « pas horrible ».

Dans ton TP

2 220 colonnes, **97,7 %** de zéros, **47** mots allumés par appel. Le premier appel ressort avec « debit card » : le TF-IDF a trouvé seul de quoi il parle.

`ngram_range=(1, 2)`

mots seuls **et** paires de mots : « thank » et « thank you »

`min_df=2`

on jette les mots vus dans un seul appel

`sublinear_tf=True`

un mot répété dix fois ne compte pas dix fois plus

4.3 Ton premier classifieur de texte

```
def modele_texte():
    return make_pipeline(
        TfidfVectorizer(ngram_range=(1, 2), min_df=2, sublinear_tf=True),
        LogisticRegression(C=1.0, max_iter=3000, class_weight="balanced"))
```

Tu dois voir : AUC proche de **0,86**, nettement au-dessus du repère « motif seul » (0,637).

`class_weight="balanced"`

25 % d'appels à 1, 75 % à 0 : sans ce réglage, le modèle répondrait « 0 » partout.

`mots_marquants`

Les poids se lisent : un poids positif tire vers « valence basse », un poids négatif vers « valence haute ».

10. Deux questions qu'on te posera

Logistique ou linéaire ?

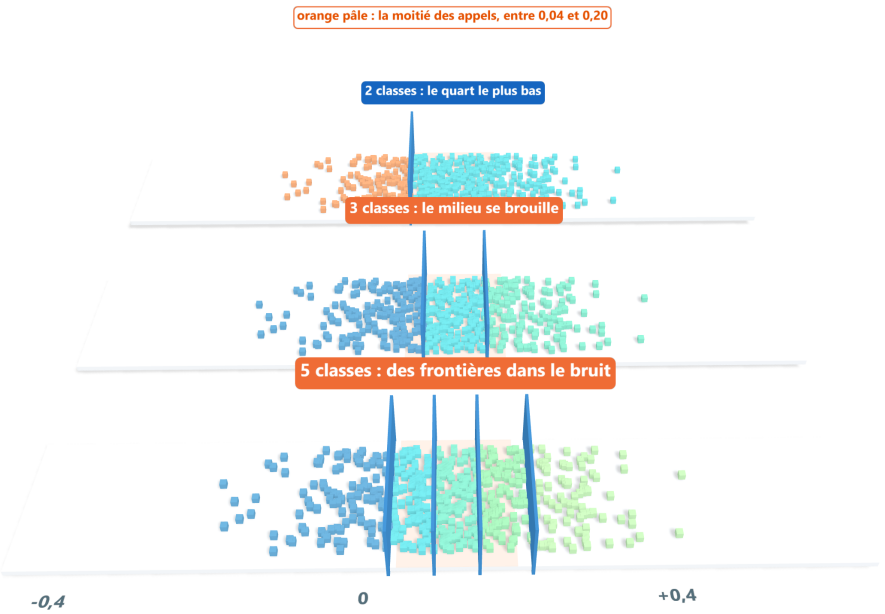
Linéaire : un nombre quelconque, pour une quantité (une durée, une valence).
Logistique : un nombre entre 0 et 1, la probabilité d'une classe. Malgré son nom, c'est un **classifieur**.

Pourquoi deux classes ?

La valence a une échelle arbitraire, la valeur est dans les extrêmes, et la décision est binaire : on rappelle le client, ou non.

Le thermomètre affiche 37,8 ; le médecin décide « fièvre » ou « pas fièvre ».

11. Et trois ou cinq classes ?



2 classes

AUC **0,849** : le quart le plus bas se reconnaît.

3 classes

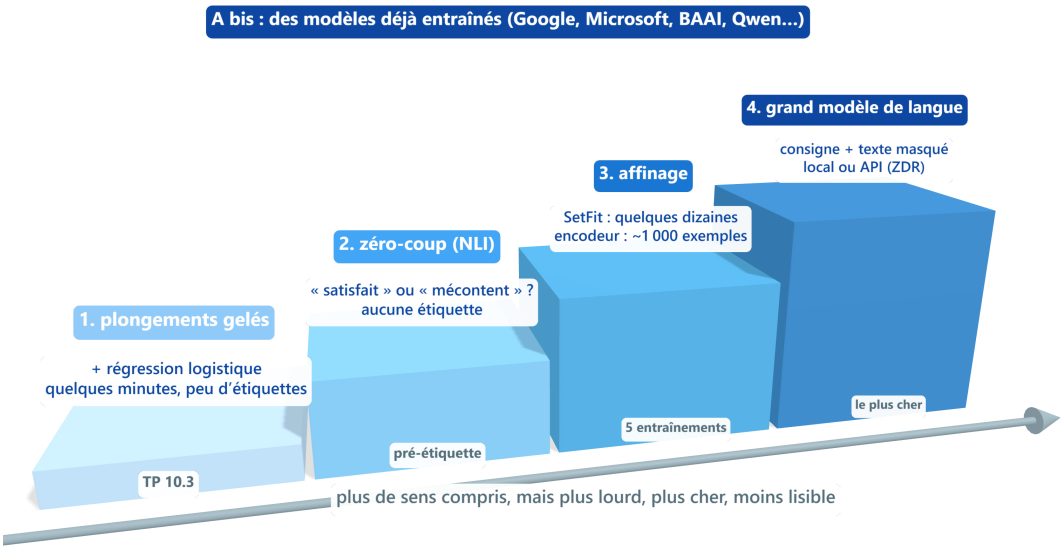
extrêmes : AUC **0,85** ; milieu : **0,66**, presque au niveau du motif seul.

5 classes

40 % de bonnes réponses ; « plutôt bas » ou « plutôt haut », pas mieux.

Ranger des enfants presque de la même taille en cinq rangs : on repère les plus grands et les plus petits, mais au milieu, c'est le hasard qui décide. Plus de rangs, ce n'est pas plus d'information, ce sont plus de frontières arbitraires.

12. La branche A bis : les modèles déjà entraînés



1. Plongements gelés (exercice 10.3)

un modèle de phrases donne un vecteur de **sens** par appel, puis la même régression logistique. Pour le français : `intfloat/multilingual-e5-base`.

2. Zéro-coup

« satisfait » ou « mécontent » ? sans aucune étiquette. Une **pré-étiquette**, pas une branche.

3. Affinage

SetFit avec quelques dizaines d'exemples ; un encodeur complet avec un millier. **Un entraînement par pli.**

4. Grand modèle de langue

une consigne + le texte **masqué**. Le plus puissant, le plus cher ; il se juge comme un outil de sentiment.

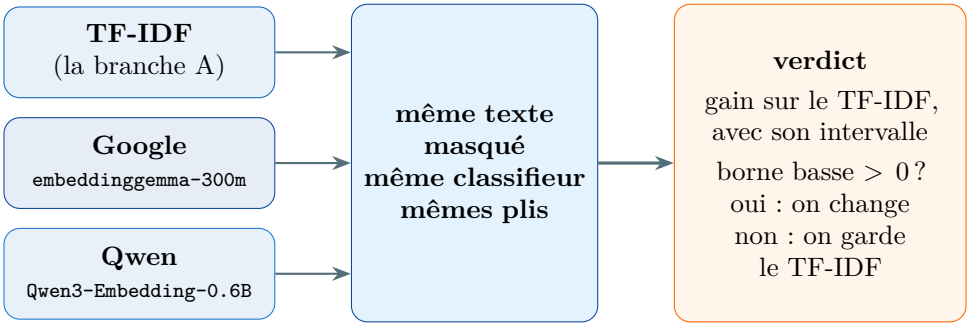
La brancher

A bis donne `scores_A2`, un score par appel, comme toutes les branches : fusion, ablation, bootstrap, sans rien changer d'autre.

La seule question : apporte-t-elle quelque chose que le TF-IDF n'a pas ?

Le TF-IDF compte les mots ; un modèle pré-entraîné a déjà lu des milliards de phrases. Il sait que « merci beaucoup » et « c'est très gentil » veulent presque dire la même chose.

Le match : TF-IDF contre Google contre Qwen



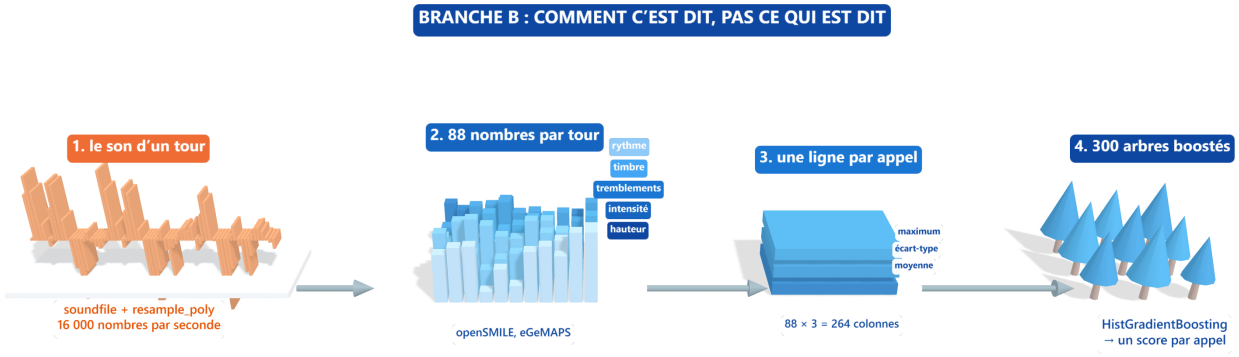
Ni « le meilleur score », ni « la moyenne ». Garder le meilleur des trois, c'est garder le lancer de dé chanceux ; la moyenne donne le même poids au médiocre qu'au bon. On change seulement si le gain est **net** (bootstrap apparié), et la **fusion apprise** pondère elle-même les représentations ; elle n'est gardée que si elle bat nettement la meilleure seule.

Pourquoi pas tout de suite un réseau de neurones ? Avec 1 446 appels, ce modèle simple est difficile à battre, il s'entraîne en une seconde, et ses poids se lisent. On ne sait si un modèle plus gros apporte quelque chose qu'en le comparant à celui-ci.

Fiche 4 : séance 3

La voix et le déroulement : deux branches qui ne lisent pas les mots.

13. Branche B : comment c'est dit



Écouter quelqu'un parler une langue qu'on ne comprend pas : on devine quand même s'il est fâché, à sa voix.

5.1 à 5.3 Du son à une ligne par appel

5.1	`signal_du_tour` : la tranche du tour, de 8 000 à 16 000 Hz	✔ 4,08 s → **65 280** nombres
5.2	`descripteurs` : openSMILE, eGeMAPS	✔ **88** valeurs par tour
5.3	`X_B` : moyenne, écart-type, maximum par appel	✔ **1 446 × 264**

hauteur	intensité	tremblements	timbre	rythme
grave ou aiguë, monocorde ou chantante	forte ou faible	jitter, shimmer : la voix qui chevrote	claire ou sourde, souffle	morceaux voisés, pauses

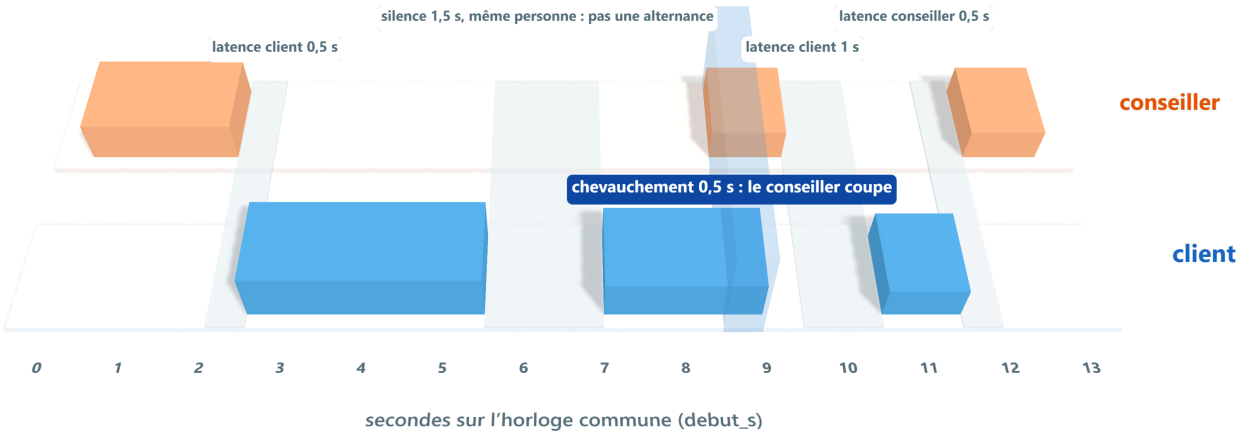
5.4 Classer, et comparer deux découpages

`HistGradientBoostingClassifier` : 300 petits arbres, chacun corrige les erreurs du précédent.

✔ **Tu dois voir :** par locuteur **0,53** au hasard **0,70**

0,70 au hasard, c'est un mirage. Les 88 descripteurs reconnaissent la personne dans **97 %** des cas (exercice 5.5) : au hasard, le modèle reconnaît des voix, pas une humeur. **Toujours séparer les personnes** entre entraînement et test.

14. Branche C : la forme de la conversation



silence total 3,5 s · alternances 4 · part du client 60 %

Regarder un match de ping-pong sans voir la balle : au rythme des coups et des pauses, on devine si la partie est fluide ou chaotique.

6.1 Les 14 indicateurs, d'abord à la main sur l'appel ci-dessus

écart = début du tour suivant − fin du tour : **positif** = silence, **négatif** = chevauchement. Deux personnes différentes = une **alternance**.

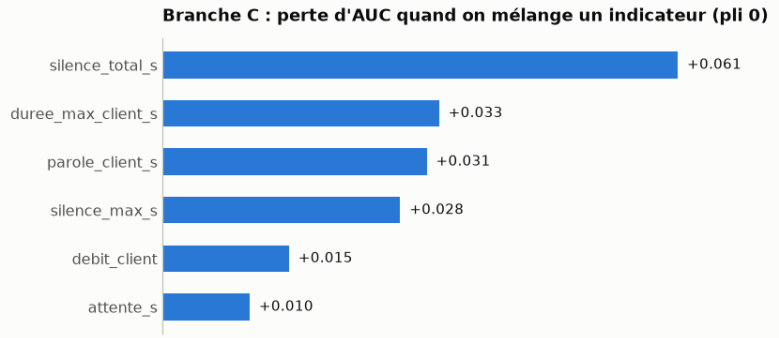
Tu dois voir : silence total **3,5 s**, alternances **4**, chevauchement **0,5 s**, part du client **0,6**, latence du client **0,75 s**.

Le piège : un silence entre deux tours de la *même* personne est un silence, mais **pas** une alternance.

6.2 et 6.3 Le tableau, puis le classifieur

14 indicateurs + attente + durée = **16 colonnes**, le même classifieur que la voix, le même découpage.

Tu dois voir : `X_C` de **1 446** × **16**, puis AUC proche de **0,61** (repère « motif seul » : 0,637).



Ce que regarde le modèle

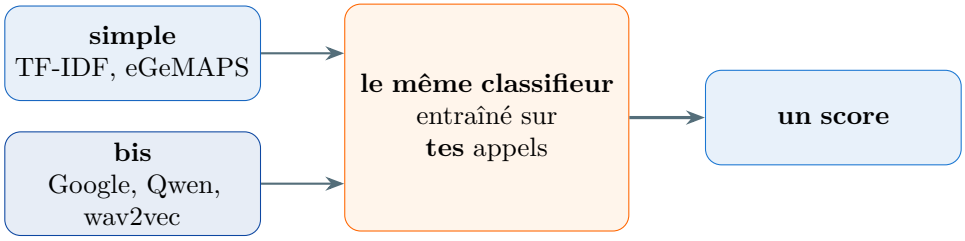
Le silence total et la longueur des tours : la **durée de la tâche**, pas l'humeur. Payer une facture fait 43 s de silences, demander les horaires 15 s.

L'importance par permutation

On mélange une colonne et on regarde si le score baisse.

On remplace un ingrédient par n'importe quoi et on goûte.

15. Pourquoi ne pas prendre directement les « bis » ?



Il faut apprendre

Un modèle de Google ne sait pas ce qu'est un client mécontent *dans cette entreprise* : le classifieur derrière s'entraîne sur tes appels, simple ou bis.

Le simple est le témoin

0,80 pour Google ne veut rien dire si le simple fait 0,79. Et seul le simple se lit.

Le bis casse plus

carte graphique, versions, jetons, licences, 512 jetons, modèle anglais. Les erreurs de données touchent les deux pareil.

Un dictionnaire très savant ne sait pas corriger ton contrôle : il faut encore lui montrer des copies corrigées.

La règle

Le simple d'abord (le repère), **le bis ensuite** (même cellule, test apparié). Le bis gagne nettement ? on le garde. Sinon, on garde le simple, sans carte graphique, avec la preuve chiffrée.